



2024.07.30 - 2024.07.31) Thorough comprehension of PPO-CLIP from ManiSkill code

按照我第一次接触CV代码的标准从头到尾注释了一遍ppo.py：

<https://github.com/YuchengWWang/basic-RL-note>

一些重点：

- Maniskill中env.step()函数逻辑

```
def step(
    self, actions: Union[Array, Dict]
) -> Tuple[Array, Array, Array, Array, Dict]:
    obs, rew, terminations, truncations, infos = self._env.step(actions)
    self.returns += rew # infos["episode"]["r"] = returns 指的是num_step步下来总的reward和

    infos["episode"] = dict(r=self.returns)

    # fix issue with gymnasium replacing truncations with a single bool
    if self.handle_truncations and self.max_episode_steps is not None:
        truncations: torch.Tensor = (
            self.base_env.elapsed_steps >= self.max_episode_steps # truncations一直是zero直到step达到max_episode_steps
        )
    if isinstance(terminations, bool):
        terminations = torch.tensor([terminations], device=self.device)
    if self.ignore_terminations:
        terminations[:] = False
    dones = torch.logical_or(terminations, truncations) # terminations:即task中定义的success判定结果, truncations:即step是否已超过num_step
    infos[
        "real_next_obs"
    ] = obs # not part of standard API but makes some RL code slightly less complicated
    if dones.any(): # 判断是否有一个返回已完成任务, 如有, 返回的info字典中加入final_info项和下划线前缀的mask
        infos["episode"]["r"] = self.returns.clone()
        final_obs = obs
        env_idx = torch.arange(0, self.num_envs, device=self.device)[dones]
        obs, _ = self.reset(options=dict(env_idx=env_idx)) # 每个epoch会跑num_step步, 这些会记录在env.elapsed_steps中但是一个epoch结束不会重置, 只有当elapsed_steps
        infos["episode"]["r"] = dones # 达到max_eps_steps时才会重置, 所以比如设置了每个epoch的num_step为20, 但是max_eps_steps为50, 就会在第三个
        infos["final_info"] = infos.copy() # epoch的第9个epoch处才出现truncation, 然后再在此处reset整个env
        # gymnasium calls it final observation but it really is just o_{t+1} or the true next observation
        infos["final_observation"] = final_obs
        # NOTE (stao): that adding masks like below is a bit redundant and not necessary
        # but this is to follow the standard gymnasium API
        infos["final_info"] = dones
        infos["final_observation"] = dones
        infos["elapsed_steps"] = dones
        # NOTE (stao): Unlike gymnasium, the code here does not add masks for every key in the info object.
    return obs, rew, terminations, truncations, infos
```

- behavior policy rollout的阶段，注意这部分的rollout相当于的完成了各个变量的在 π_{old} 下的采样，后面在做PG update的时候通过重要性采样比率加权不需要重新rollout

```
rollout_time = time.time()
# 从这里开始，根据旧的策略进行采样 x~ $\pi_{old}$ ，这时候从agent.get_action_and_value(obs)得到的logprob是旧策略下的概率密度，我们记录为p_old(x)，
# 相对的这些从旧策略采出来的x在新策略分布下的概率密度记录为p_new(x)
for step in range(0, args.num_steps):
    global_step += args.num_envs
    obs[step] = next_obs
    dones[step] = next_done

    # ALGO LOGIC: action logic
    with torch.no_grad():
        # 从代码看来这个logprob是每个平行环境下，从各个独立的正态分布采样得到的动作值反求其对数概率密度的求和，比如这里相当于有512*8个正态分布采样
        # 到了512*8个动作值，每个环境下有8个动作变量，分别按照他们对应的正态分布反求概率密度取对数再对这8个值求和，最后logprob是512维的
        action, logprob, _, value = agent.get_action_and_value(next_obs)
        values[step] = value.flatten()
    actions[step] = action
    logprobs[step] = logprob

    # TRY NOT TO MODIFY: execute the game and log data.
    next_obs, reward, terminations, truncations, infos = envs.step(clip_action(action))
    next_done = torch.logical_or(terminations, truncations).to(torch.float32)
    rewards[step] = reward.view(-1)

    if "final_info" in infos:
        rollout_time = time.time() - rollout_time
```

- 递归法计算广义优势函数GAE，计算得到A的时候顺带反求了rollout过程中的returns(注意此前的rollout过程中t只记录了reward，那里面虽然有[self.returns += rew]这个变量但这个是没有折扣加权的所以后面并不是使用这个)。
- returns = advantages + values, 原本的优势函数定义是 $A_t = Q_t - V_t$, 这当中的 Q_t 是要即刻奖励和折扣后的后续奖励求和得到的(即 G_t),我们要求的其实是累计回报 G_t 而不是即刻奖励 R_t ，所以这里的returns含义是 G_t 而不是 R_t (R_t 是reward)，这个returns在后面作为价值函数实际值的蒙特卡洛估计值。
- KL散度的计算和另一种低方差的无偏估计算法：

```
# 在得到旧策略下采出来的Trajectory: (st, at, rt)后，可以进行update_epochs次梯度更新
# 在下面一个epoch时agent的参数还没被更新，所以newlogprob和logprob是一致的，当第一次梯度更新后，就有了新的策略网络，我们也能算出在旧策略采出来的x在新策略下的概率密度p_new(x)，x~p_old
# 于是乎下面的logratio计算的就是log[p_new(x)/p_old(x)]，而KL[p_old||p_new] = E{log[p_old(x)/p_new(x)]}，where x~p_old，所以传统的KL散度就是下面的old_approx_kl
for epoch in range(args.update_epochs):
    np.random.shuffle(b_inds)
    for start in range(0, args.batch_size, args.minibatch_size):
        end = start + args.minibatch_size
        mb_inds = b_inds[start:end]

        _, newlogprob, entropy, newvalue = agent.get_action_and_value(b_obs[mb_inds], b_actions[mb_inds]) # [
        logratio = newlogprob - b_logprobs[mb_inds]
        ratio = logratio.exp()

        with torch.no_grad():
            # calculate approx_kl http://joschu.net/blog/kl-approx.html
            old_approx_kl = (-logratio).mean() # 传统的KL散度
            approx_kl = ((ratio - 1) - logratio).mean() # 优化后的KL散度表达式，具有更小的方差同时还是无偏估计，证明见上面的网址链接
```

- PPO-CLIP 损失函数构成逻辑（非常重要）

1. 其实原来的目标是最大化动作价值，然后演化为最大化优势函数，但是优势函数的计算是on policy的 $E_{A \sim \pi_{\text{new}}}[f]$ ，我们在更新 π_{new} 的同时又要用它的采样 $A(\text{behavior policy})$ ，相当于下面每次`optimizer.step()`之后都要重新像上面一样走一遍rollout，效率很低，而重要性采样比率加权的作用就是把on policy的 $E_{A \sim \pi_{\text{new}}}[f]$ 变成 off policy的 $E_{A \sim \pi_{\text{old}}}[\text{ratio} * f]$ ，提高效率
2. 价值损失函数是在最小化实际值函数和估计值函数之间的MSE，这个实际值函数其实是无法得到的，但是我们可以用蒙特卡罗的方法找到这个实际值函数的估计值，值函数的定义是在给定策略的情况下，从状态s出发得到一系列episode的returns的期望，那这个在前面进行rollout的时候我们已经记录了旧策略下从各s出发得到的rewards，又在计算advantage的时候顺便反求了returns，这个returns是每个s都配有一个的，也就是真实值函数的蒙特卡洛估计值

```
# Policy loss 策略损失函数是在最大化“重要性采样比率加权的优势函数”
# 其实原来的目标是最大化动作价值，然后演化为最大化优势函数，但是优势函数的计算是on policy的 $E_{A \sim \pi_{\text{new}}}[f]$ ，我们在更新 $\pi_{\text{new}}$ 的同时又要用它的采样 $A(\text{behavior policy})$ ，
# 相当于下面每次optimizer.step()之后都要重新像上面一样走一遍rollout，效率很低，
# 而重要性采样比率加权的作用就是把on policy的 $E_{A \sim \pi_{\text{new}}}[f]$ 变成 off policy的 $E_{A \sim \pi_{\text{old}}}[\text{ratio} * f]$ ，提高效率
pg_loss1 = -mb_advantages * ratio
pg_loss2 = -mb_advantages * torch.clamp(ratio, 1 - args.clip_coef, 1 + args.clip_coef)
pg_loss = torch.max(pg_loss1, pg_loss2).mean() # PPO的Clipped Surrogate Objective版本 (PPO-Clip)，以下是 $L^{\text{CLIP}}$ 的表达式

# Value loss 价值损失函数是在最小化实际值函数和估计值函数之间的MSE，这个实际值函数其实是无法得到的，但是我们可以用蒙特卡罗的方法找到这个实际值函数的估计值，
# 值函数的定义是在给定策略的情况下，从状态s出发得到一系列episode的returns的期望，那这个在前面进行rollout的时候我们已经记录了旧策略下从各s出发得到的rewards，
# 又在计算advantage的时候顺便反求了returns，这个returns是每个s都配有一个的，也就是真实值函数的蒙特卡洛估计值
newvalue = newvalue.view(-1)
if args.clip_vloss:
    v_loss_unclipped = (newvalue - b_returns[mb_inds]) ** 2
    v_clipped = b_values[mb_inds] + torch.clamp(
        newvalue - b_values[mb_inds],
        -args.clip_coef,
        args.clip_coef,
    )
    v_loss_clipped = (v_clipped - b_returns[mb_inds]) ** 2
    v_loss_max = torch.max(v_loss_unclipped, v_loss_clipped)
    v_loss = 0.5 * v_loss_max.mean()
else:
    v_loss = 0.5 * ((newvalue - b_returns[mb_inds]) ** 2).mean() #  $L^{\text{VF}}(\theta_v) = (V(\theta_v) - g)^2$ 

entropy_loss = entropy.mean()
#  $L(\theta) = L^{\text{CLIP}}(\theta_p) + c1 * L^{\text{VF}}(\theta_v) - c2 * H(\theta_p)$ ，最后一项是熵损失，目标是最大化策略的熵，鼓励策略产生更多的不确定性
loss = pg_loss - args.ent_coef * entropy_loss + v_loss * args.vf_coef
```